

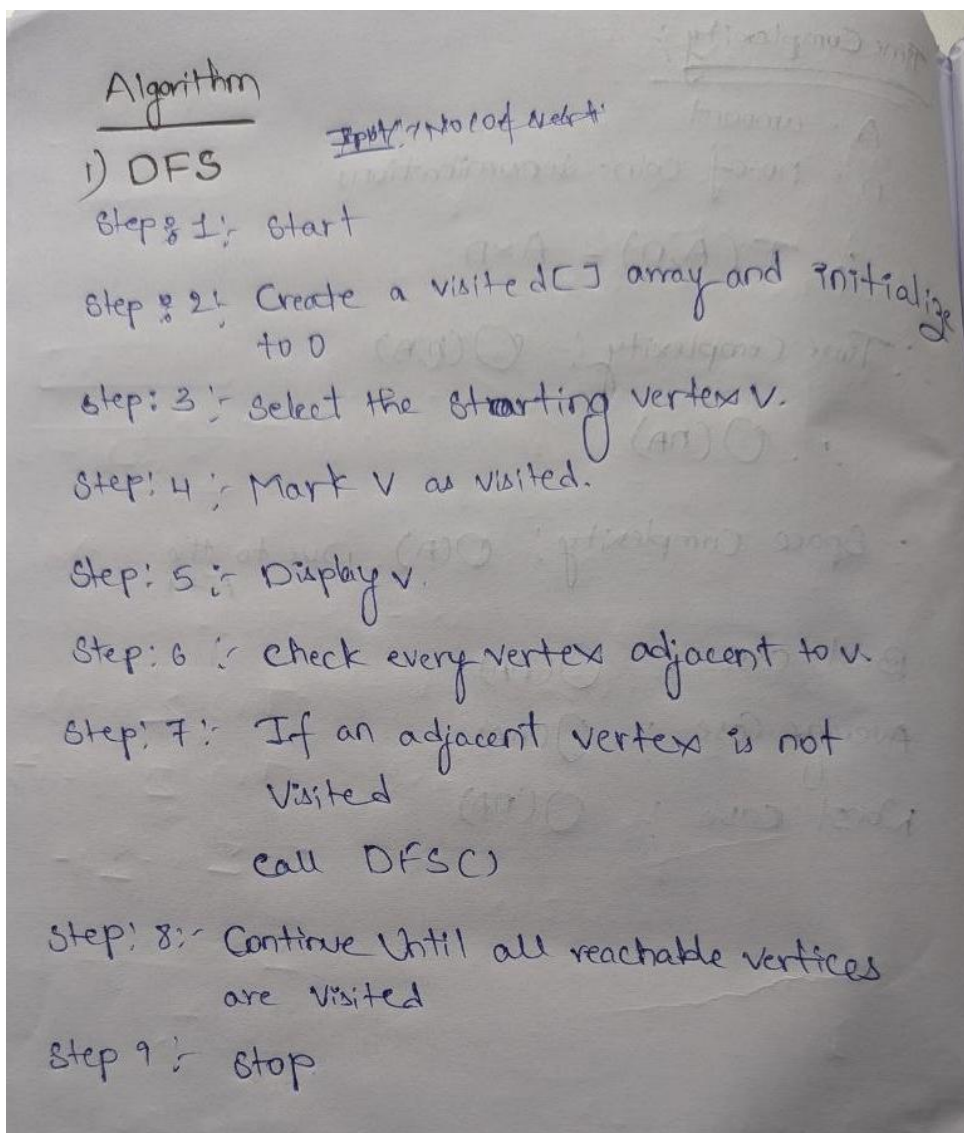
Practical: 8

Implementation of Graph and Searching (DFS and BFS).

22/09/2026

Aim: To implement of Graph and Searching (DFS and BFS).

Algorithm:





2) BFS

Step 1 :- start

Step 2 :- Create visited[] = 1

Step 3 :- Insert V into queue

Step 4 :- while queue is not empty

 V = remove front element

 Display V

 for i = 0 to V-1

 If graph[V][i] == 1 and

 visited[i] == 0

 Do

 visited[i] = 1

 Insert i into queue

Step 5 :- stop.

Program:

```
#include <iostream>
#include <queue>
using namespace std;

int main()
{
    // Static adjacency matrix
    int graph[5][5] =
    {
        {0, 1, 1, 0, 0},
        {1, 0, 0, 1, 1},
        {1, 0, 0, 0, 0},
        {0, 1, 0, 0, 0},
        {0, 1, 0, 0, 0}
    };

    int visited[5] = {0};

    // ----- DFS -----
    cout << "DFS Traversal: ";

    int stack[5];
    int top = -1;

    stack[++top] = 0;
    visited[0] = 1;

    while (top >= 0)
    {
        int vertex = stack[top--];
        cout << vertex << " ";

        // Add adjacent vertices
        for (int i = 4; i >= 0; i--)
        {
            if (graph[vertex][i] == 1 && visited[i] == 0)
            {
                visited[i] = 1;
                stack[++top] = i;
            }
        }
    }

    // Reset visited array
```

```
for (int i = 0; i < 5; i++)
    visited[i] = 0;

// ----- BFS -----
cout << "\nBFS Traversal: ";

queue<int> q;

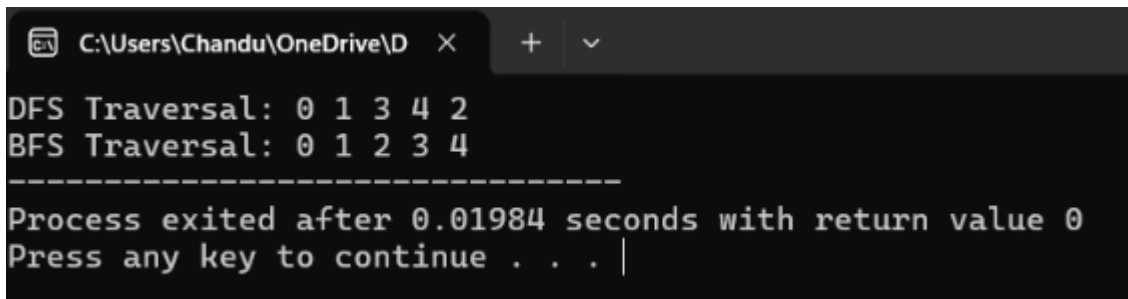
q.push(0);
visited[0] = 1;

while (!q.empty())
{
    int vertex = q.front();
    q.pop();

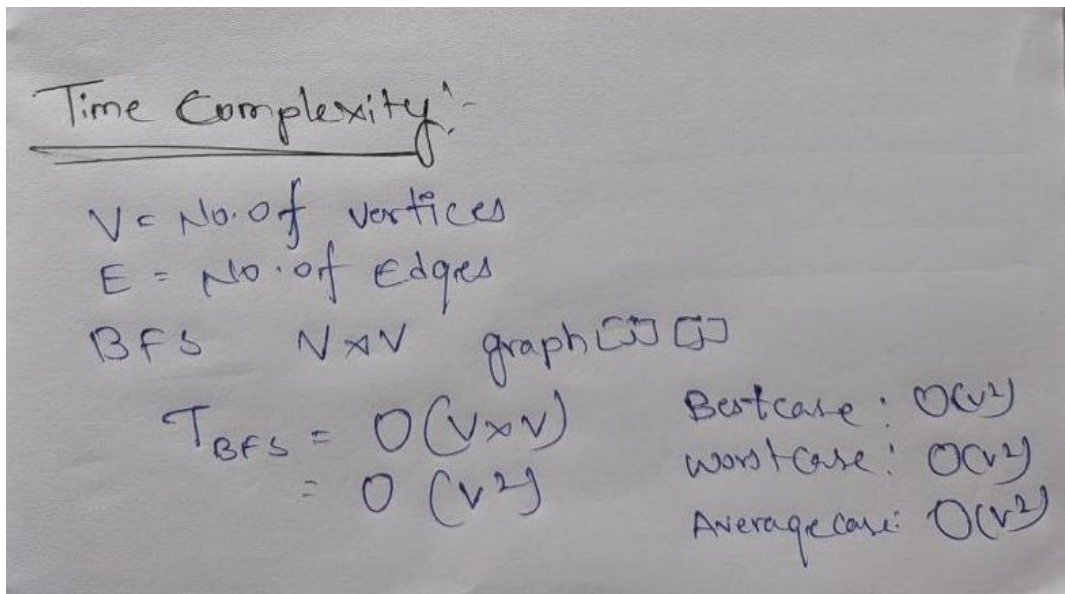
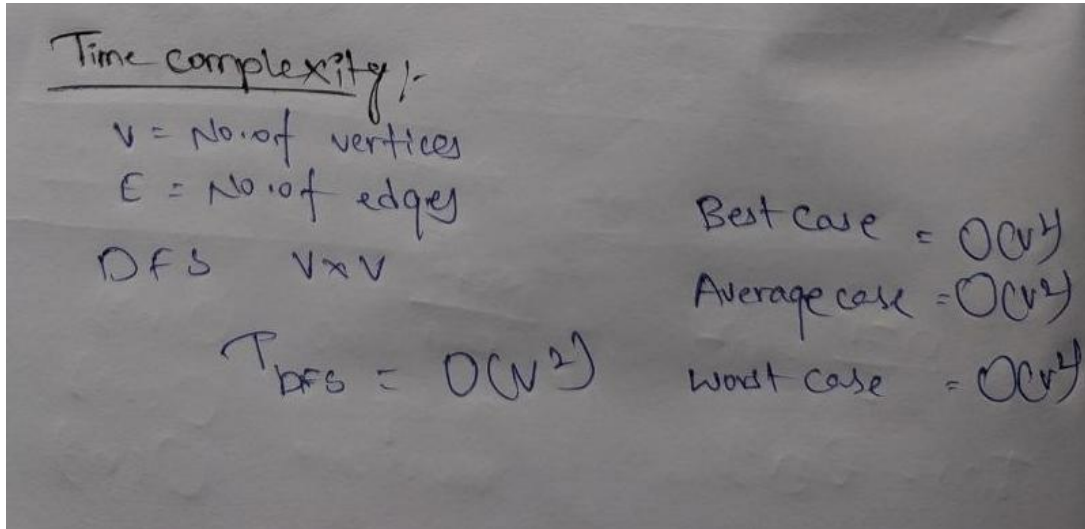
    cout << vertex << " ";

    for (int i = 0; i < 5; i++)
    {
        if (graph[vertex][i] == 1 && visited[i] == 0)
        {
            visited[i] = 1;
            q.push(i);
        }
    }
}

return 0;
}
```

Output:

Time Complexity:



Conclusion:

The Implementation of Graph and Searching DFS and BFS are successfully executed with the time complexity of $O(V^2)$

And same for all the three cases of best, average and worst.